

Program 1

1. **write the sql command for Creation, altering and dropping of student table and inserting rows into a table (use constraints while creating tables) examples using SELECT command.**

1. Creation of the department table

```
CREATE TABLE department (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(10) NOT NULL,  
    location VARCHAR(10)  
);
```

```
desc department;
```

Name	Null?	Type
-----	-----	-----
DEPARTMENT_ID	NOT NULL	NUMBER(38)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(10)
LOCATION		VARCHAR2(10)

Creation of the student table

```
CREATE TABLE student (  
    student_id INT PRIMARY KEY,  
    first_name VARCHAR(10) NOT NULL,  
    last_name VARCHAR(10) NOT NULL,  
    email VARCHAR(15) UNIQUE,  
    date_of_birth DATE,  
    department_id INT,  
    CONSTRAINT fk_department FOREIGN KEY (department_id)  
        REFERENCES department(department_id)  
);
```

```
desc student;
```

Name	Null?	Type
-----	-----	-----
STUDENT_ID	NOT NULL	NUMBER(38)
FIRST_NAME	NOT NULL	VARCHAR2(10)
LAST_NAME	NOT NULL	VARCHAR2(10)
EMAIL		VARCHAR2(15)
DATE_OF_BIRTH		DATE
DEPARTMENT_ID		NUMBER(38)

Inserting rows into the department table

```
INSERT INTO department VALUES (101, 'cse', 'S Block');  
INSERT INTO department VALUES (102, 'ece', 'R Block'),  
INSERT INTO department VALUES (103, 'CIVIL', 'V Block');
```

Displaying department contents:

```
Select * from department;
```

DEPARTMENT_ID	DEPARTMENT	LOCATION
101	cse	S Block
102	ece	R Block
103	CIVIL	V Block

Inserting rows into the student table

```
INSERT INTO student VALUES (1, 'John', 'Doe', 'john@gmail.com', TO_DATE('2000-05-15', 'YYYY-MM-DD'), 101);
INSERT INTO student VALUES (2, 'Jane', 'Smith', 'jane@gmail.com', TO_DATE('2001-08-22', 'YYYY-MM-DD'), 102);
INSERT INTO student VALUES (3, 'Alice', 'John', 'alice@gmail.com', TO_DATE('1999-12-30', 'YYYY-MM-DD'), 101);
```

SELECT command

```
SELECT * FROM student;
```

STUDENT_ID	FIRST_NAME	LAST_NAME	EMAIL	DATE_OF_B	DEPARTMENT_ID
1	John	Doe	john@gmail.com	15-MAY-00	101
2	Jane	Smith	jane@gmail.com	22-AUG-01	102
3	Alice	John	alice@gmail.com	30-DEC-99	101

Select specific columns

```
SELECT first_name, last_name, email FROM student;
```

FIRST_NAME	LAST_NAME	EMAIL
John	Doe	john@gmail.com
Jane	Smith	jane@gmail.com
Alice	John	alice@gmail.com

Select with a condition

```
SELECT * FROM student
WHERE department_id = 101;
```

STUDENT_ID	FIRST_NAME	LAST_NAME	EMAIL	DATE_OF_B	DEPARTMENT_ID
1	John	Doe	john@gmail.com	15-MAY-00	101
3	Alice	John	alice@gmail.com	30-DEC-99	101

Select with sorting

```
SELECT * FROM student
ORDER BY last_name ASC;
```

STUDENT_ID	FIRST_NAME	LAST_NAME	EMAIL	DATE_OF_B	DEPARTMENT_ID
1	John	Doe	john@gmail.com	15-MAY-00	101
3	Alice	John	alice@gmail.com	30-DEC-99	101
2	Jane	Smith	jane@gmail.com	22-AUG-01	102

Select with aggregation

```
SELECT department_id, COUNT(*) AS student_count
FROM student
GROUP BY department_id;
```

DEPARTMENT_ID	STUDENT_COUNT
101	2
102	1

Altering the student table

Add a new column

```
ALTER TABLE student ADD phone_number VARCHAR2(15);
```

Desc student;

Name	Null?	Type
STUDENT_ID	NOT NULL	NUMBER(38)
FIRST_NAME	NOT NULL	VARCHAR2(10)
LAST_NAME	NOT NULL	VARCHAR2(10)
EMAIL		VARCHAR2(15)
DATE_OF_BIRTH		DATE
DEPARTMENT_ID		NUMBER(38)
PHONE_NUMBER		VARCHAR2(15)

Modify an existing column

```
ALTER TABLE department MODIFY (location VARCHAR2(12));
desc department;
```

Name	Null?	Type
DEPARTMENT_ID	NOT NULL	NUMBER(38)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(10)
LOCATION		VARCHAR2(12)

Drop a column

```
ALTER TABLE student DROP COLUMN phone_number;
```

Desc student;

Name	Null?	Type
STUDENT_ID	NOT NULL	NUMBER(38)
FIRST_NAME	NOT NULL	VARCHAR2(10)
LAST_NAME	NOT NULL	VARCHAR2(10)
EMAIL		VARCHAR2(15)
DATE_OF_BIRTH		DATE
DEPARTMENT_ID		NUMBER(38)

Program 2:

Create a database and queries (along with sub Queries) using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, INTERSET, Constraints. Example:- Select the roll number and name of the student who secured fourth rank in the class.

```
CREATE TABLE Students (  
  RollNumber INT PRIMARY KEY,  
  Name VARCHAR(10) NOT NULL,  
  Class INT CHECK (Class BETWEEN 1 AND 12),  
  Section CHAR(1) NOT NULL,  
  Age INT CHECK (Age BETWEEN 5 AND 18)  
);
```

Desc Students;

```
SQL> desc students;
```

Name	Null?	Type
-----	-----	-----
ROLLNUMBER	NOT NULL	NUMBER(38)
NAME	NOT NULL	VARCHAR2(10)
CLASS		NUMBER(38)
SECTION	NOT NULL	CHAR(1)
AGE		NUMBER(38)

```
CREATE TABLE Marks (  
  RollNumber INT,  
  Subject VARCHAR(15),  
  MarksObtained INT CHECK (MarksObtained BETWEEN 0 AND 100),  
  PRIMARY KEY (RollNumber, Subject),  
  FOREIGN KEY (RollNumber) REFERENCES Students(RollNumber)  
);
```

Desc Marks;

Name	Null?	Type
-----	-----	-----
ROLLNUMBER	NOT NULL	NUMBER(38)
SUBJECT	NOT NULL	VARCHAR2(15)
MARKSOBTAINED		NUMBER(38)

```
CREATE TABLE Ranks (  
  RollNumber INT PRIMARY KEY,  
  Rank INT NOT NULL,  
  FOREIGN KEY (RollNumber) REFERENCES Students(RollNumber)  
);
```

desc Ranks;

Name	Null?	Type
-----	-----	-----
ROLLNUMBER	NOT NULL	NUMBER(38)
RANK	NOT NULL	NUMBER(38)

2. Inserting Sample Data

```
INSERT INTO Students VALUES(1, 'Alice', 10, 'A', 15);
INSERT INTO Students VALUES (2, 'Bob', 10, 'A', 16);
INSERT INTO Students VALUES(3, 'Charlie', 10, 'A', 15);
INSERT INTO Students VALUES(4, 'David', 10, 'A', 15);
INSERT INTO Students VALUES(5, 'Eve', 10, 'A', 16);
```

```
select * from Students;
```

ROLLNUMBER	NAME	CLASS	S	AGE
1	Alice	10	A	15
2	Bob	10	A	16
3	Charlie	10	A	15
4	David	10	A	15
5	Eve	10	A	16

```
INSERT INTO Marks VALUES (1, 'Math', 95);
INSERT INTO Marks VALUES(1, 'Science', 90);
INSERT INTO Marks VALUES (2, 'Math', 88);
INSERT INTO Marks VALUES (2, 'Science', 84);
INSERT INTO Marks VALUES (3, 'Math', 78);
INSERT INTO Marks VALUES (3, 'Science', 80);
INSERT INTO Marks VALUES (4, 'Math', 92);
INSERT INTO Marks VALUES (4, 'Science', 89);
INSERT INTO Marks VALUES (5, 'Math', 85);
INSERT INTO Marks VALUES (5, 'Science', 87);
```

```
Select * from Marks;
```

ROLLNUMBER	SUBJECT	MARKSOBTAINED
1	Math	95
1	Science	90
2	Math	88
2	Science	84
3	Math	78
3	Science	80
4	Math	92
4	Science	89
5	Math	85
5	Science	87

10 rows selected.

```
INSERT INTO Ranks (RollNumber, Rank) VALUES(1, 1);
INSERT INTO Ranks (RollNumber, Rank) VALUES (4, 2);
INSERT INTO Ranks (RollNumber, Rank) VALUES (2, 3);
INSERT INTO Ranks (RollNumber, Rank) VALUES (3, 4);
INSERT INTO Ranks (RollNumber, Rank) VALUES (5, 5);
```

```
Select * from Ranks;
```

ROLLNUMBER	RANK
1	1
4	2
2	3
3	4
5	5

3. Queries with Constraints and Operations

```
SELECT RollNumber, Name
FROM Students
WHERE RollNumber = (SELECT RollNumber FROM Ranks WHERE Rank = 4);
```

ROLLNUMBER	NAME
3	Charlie

B. Query using IN

```
SELECT Name
FROM Students
WHERE RollNumber IN (SELECT RollNumber FROM Ranks WHERE Rank <= 3);
```

NAME
Alice
Bob
David

C. Query using EXISTS

```
SELECT Name
FROM Students S
WHERE EXISTS (
  SELECT 1
  FROM Marks M
  WHERE S.RollNumber = M.RollNumber AND M.MarksObtained > 90
);
```

NAME
Alice
David

D. Query using NOT EXISTS

```
SELECT Name
FROM Students S
WHERE NOT EXISTS (
```

```
SELECT 1
FROM Marks M
WHERE S.RollNumber = M.RollNumber AND M.Subject = 'Math' AND M.MarksObtained <
85
);
```

```
NAME
-----
Alice
Bob
David
Eve
```

E. Query using ANY

```
SELECT Name
FROM Students
WHERE Age > ANY (SELECT Age FROM Students WHERE Section = 'A');
```

```
NAME
-----
Bob
Eve
```

F. Query using ALL

```
SELECT Name
FROM Students
WHERE Age <= ALL (SELECT Age FROM Students WHERE Section = 'A');
```

```
NAME
-----
David
Alice
Charlie
```

G. Query using UNION

```
SELECT Name
FROM Students
WHERE Class = 10
UNION
SELECT Name
FROM Students
WHERE Age > 15;
```

```
NAME
-----
Alice
Bob
Charlie
David
Eve
```

H. Query using INTERSECT

```
SELECT Name
FROM Students
WHERE Age > 15
INTERSECT
SELECT Name
FROM Students
WHERE Section = 'A';
```

```
NAME
-----
Bob
Eve
```

Program 3:

create a company database and write the Queries using Aggregate functions (COUNT, SUM, AVG, MAX and MIN), GROUP BY, HAVING and Creation and dropping of Views.

```
CREATE TABLE Departments (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(20),  
    location VARCHAR(20)  
);
```

```
desc Departments;
```

Name	Null?	Type
-----	-----	-----
DEPARTMENT_ID	NOT NULL	NUMBER(38)
DEPARTMENT_NAME		VARCHAR2(20)
LOCATION		VARCHAR2(20)

```
CREATE TABLE Employees (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(20),  
    department_id INT,  
    salary DECIMAL(10,2),  
    hire_date DATE,  
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)  
);
```

```
desc Employees;
```

Name	Null?	Type
-----	-----	-----
EMPLOYEE_ID	NOT NULL	NUMBER(38)
EMPLOYEE_NAME		VARCHAR2(20)
DEPARTMENT_ID		NUMBER(38)
SALARY		NUMBER(10,2)
HIRE_DATE		DATE

```
CREATE TABLE Projects  
(  
    project_id INT PRIMARY KEY,  
    project_name VARCHAR(20),  
    department_id INT,  
    budget DECIMAL(15,2),  
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)  
);
```

```
desc Projects;
```

```
SQL> desc Projects;
```

Name	Null?	Type
-----	-----	-----
PROJECT_ID	NOT NULL	NUMBER(38)
PROJECT_NAME		VARCHAR2(20)
DEPARTMENT_ID		NUMBER(38)
BUDGET		NUMBER(15,2)

```
INSERT INTO Departments VALUES (1, 'HR', 'New York');
```

```
1 row created.
```

```
INSERT INTO Departments VALUES (2, 'IT', 'San Francisco');
```

```
1 row created.
```

```
INSERT INTO Departments VALUES (3, 'Finance', 'Chicago');
```

```
1 row created.
```

```
Select * from Departments;
```

```
SQL> select * from Departments;
```

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION
1	HR	New York
2	IT	San Francisco
3	Finance	Chicago

```
INSERT INTO Employees VALUES (101, 'John Doe', 1, 50000, TO_DATE('2000-05-15', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
INSERT INTO Employees VALUES (102, 'Jane Smith', 1, 55000, TO_DATE('2019-03-22', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
INSERT INTO Employees VALUES (103, 'Mike Johnson', 2, 75000, TO_DATE('2021-07-10', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
INSERT INTO Employees VALUES (104, 'Emily Davis', 2, 80000, TO_DATE('2020-05-18', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
INSERT INTO Employees VALUES (105, 'David Lee', 2, 90000, TO_DATE('2018-11-30', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
INSERT INTO Employees VALUES (106, 'Sarah Brown', 3, 65000, TO_DATE('2022-02-09', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
Select * from Employees;
```

```
SQL> select * from Employees;
```

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT_ID	SALARY	HIRE_DATE
101	John Doe	1	50000	15-MAY-00
102	Jane Smith	1	55000	22-MAR-19
103	Mike Johnson	2	75000	10-JUL-21
104	Emily Davis	2	80000	18-MAY-20
105	David Lee	2	90000	30-NOV-18
106	Sarah Brown	3	65000	09-FEB-22

```
INSERT INTO Projects VALUES (501, 'Recruitment Drive', 1, 100000);
```

```
1 row created.
```

```
INSERT INTO Projects VALUES (502, 'Network Upgrade', 2, 250000);
```

```
1 row created.
```

```
INSERT INTO Projects VALUES (503, 'Financial Audit', 3, 150000);
```

```
1 row created.
```

```
Select * from Projects;
```

```
SQL> Select * from Projects;
```

PROJECT_ID	PROJECT_NAME	DEPARTMENT_ID	BUDGET
501	Recruitment Drive	1	100000
502	Network Upgrade	2	250000
503	Financial Audit	3	150000

Aggregate Queries with GROUP BY

1. Number of employees per department

```
SELECT Department_ID, COUNT (*) AS TotalEmployees  
FROM Employees  
GROUP BY Department_ID;
```

DEPARTMENT_ID	TOTALEMPLOYEES
1	2
2	3
3	1

2. Total salary expenditure per department

```
SELECT department_id, SUM(salary) AS total_salary  
FROM Employees  
GROUP BY department_id;
```

DEPARTMENT_ID	TOTALSALARY
1	105000
2	245000
3	65000

3. Average salary in each department

```
SELECT department_id, AVG(salary) AS average_salary  
FROM Employees  
GROUP BY department_id;
```

DEPARTMENT_ID	AVERAGE_SALARY
1	52500
2	81666.6667
3	65000

4. Highest and lowest salaries per department

```
SELECT department_id,
       MAX(salary) AS highest_salary,
       MIN(salary) AS lowest_salary
FROM Employees
GROUP BY department_id;
```

DEPARTMENT_ID	HIGHEST_SALARY	LOWEST_SALARY
1	55000	50000
2	90000	75000
3	65000	65000

HAVING Clause Examples

1. Departments with more than 2 employees

```
SELECT department_id, COUNT(*) AS employee_count
FROM Employees
GROUP BY department_id
HAVING COUNT(*) > 2;
```

DEPARTMENT_ID	EMPLOYEE_COUNT
2	3

2. Departments with average salary above \$70,000

```
SELECT department_id, AVG(salary) AS avg_salary
FROM Employees
GROUP BY department_id
HAVING AVG(salary) > 70000;
```

DEPARTMENT_ID	AVG_SALARY
2	81666.6667

Create Views

1. Department Summary View

```
CREATE VIEW DepartmentSummary AS
SELECT d.department_name,
       COUNT(e.employee_id) AS employee_count,
       SUM(e.salary) AS total_salary,
       AVG(e.salary) AS average_salary,
       MAX(e.salary) AS highest_salary,
       MIN(e.salary) AS lowest_salary
```

```
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name;
```

```
View created.
```

```
SELECT * FROM DepartmentSummary;
```

DEPARTMENT_NAME	EMPLOYEE_COUNT	TOTAL_SALARY	AVERAGE_SALARY	HIGHEST_SALARY	LOWEST_SALARY
HR	2	105000	52500	55000	50000
IT	3	245000	81666.6667	90000	75000
Finance	1	65000	65000	65000	65000

2.Project Details View

```
CREATE VIEW ProjectDetails AS
SELECT p.project_name,
       d.department_name,
       p.budget
FROM Projects p
JOIN Departments d ON p.department_id = d.department_id;
```

```
View created.
```

Query the Views

```
SELECT * FROM ProjectDetails;
```

PROJECT_NAME	DEPARTMENT_NAME	BUDGET
Recruitment Drive	HR	100000
Network Upgrade	IT	250000
Financial Audit	Finance	150000

Drop a View

```
DROP VIEW ProjectDetails;
```

```
SQL> DROP VIEW ProjectDetails;
```

```
View dropped.
```

Program 4

create a table and Queries using Conversion functions (to_char, to_number and to_date), string functions (Concatenation, lpad, rpad, ltrim, rtrim, lower, upper, initcap, length, substr and instr), date functions (Sysdate, next_day, add_months, last_day, months_between, least, greatest, trunc, round, to_char, to_date).

```
CREATE TABLE employees (  
    employee_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(10),  
    last_name VARCHAR2(10),  
    hire_date DATE,  
    salary VARCHAR2(10), -- Stored as string for demonstration  
    email VARCHAR2(100)  
);
```

```
INSERT INTO employees (employee_id, first_name, last_name, hire_date, salary,  
email)  
VALUES (1, 'John', 'Doe', TO_DATE('2023-01-15', 'YYYY-MM-DD'), '5000',  
'john.doe@example.com');
```

```
INSERT INTO employees (employee_id, first_name, last_name, hire_date, salary,  
email)  
VALUES (2, 'Jane', 'Smith', TO_DATE('2022-11-20', 'YYYY-MM-DD'), '7500',  
'jane.smith@example.com');
```

```
INSERT INTO employees (employee_id, first_name, last_name, hire_date, salary,  
email)  
VALUES (3, 'Alice', 'Johnson', TO_DATE('2023-05-10', 'YYYY-MM-DD'), '6000',  
'alice.johnson@example.com');
```

Conversion functions

(a) to_char

```
SELECT employee_id, first_name, TO_CHAR(hire_date, 'DD-Mon-YYYY') AS  
formatted_hire_date  
FROM employees;
```

Output:

EMPLOYEE_ID	FIRST_NAME	FORMATTED_HIRE_DATE
1	John	15-Jan-2023
2	Jane	20-Nov-2022
3	Alice	10-May-2023

(b) `to_number`

```
SELECT employee_id, first_name, TO_NUMBER(salary) AS numeric_salary
FROM employees;
```

Output:

EMPLOYEE_ID	FIRST_NAME	NUMERIC_SALARY
1	John	5000
2	Jane	7500
3	Alice	6000

(c) `to_date`

```
SELECT employee_id, first_name, hire_date
FROM employees
WHERE hire_date = TO_DATE('2023-01-15', 'YYYY-MM-DD');
```

Output:

EMPLOYEE_ID	FIRST_NAME	HIRE_DATE
1	John	15-JAN-23

4. Queries Using String Functions

a. Concatenation

```
SELECT employee_id, first_name || ' ' || last_name AS full_name
FROM employees;
```

Output:

EMPLOYEE_ID	FULL_NAME
1	John Doe
2	Jane Smith
3	Alice Johnson

b. LPAD and RPAD

```
SELECT employee_id, LPAD(first_name, 10, '*') AS lpad_name, RPAD(last_name, 10, '-') AS rpad_name
FROM employees;
```

Output:

EMPLOYEE_ID	LPAD_NAME	RPAD_NAME
1	*****John	Doe-----
2	*****Jane	Smith-----
3	*****Alice	Johnson---

c. LTRIM and RTRIM

```
SELECT employee_id, LTRIM(first_name, 'J') AS ltrim_name, RTRIM(last_name, 'n') AS rtrim_name
FROM employees;
```

Output:

EMPLOYEE_ID	LTRIM_NAME	RTRIM_NAME
1	ohn	Doe
2	ane	Smith
3	Alice	Johnson

d. LOWER, UPPER, and INITCAP

```
SELECT employee_id, LOWER(first_name) AS lower_name, UPPER(last_name) AS upper_name, INITCAP(email) AS formatted_email
FROM employees;
```

Output:

EMPLOYEE_ID	LOWER_NAME	UPPER_NAME	FORMATTED_EMAIL
1	john	DOE	John.Doe@Example.Com
2	jane	SMITH	Jane.Smith@Example.Com
3	alice	JOHNSON	Alice.Johnson@Example.Com

e. LENGTH

```
SELECT employee_id, first_name, LENGTH(first_name) AS name_length
FROM employees;
```

Output:

EMPLOYEE_ID	FIRST_NAME	NAME_LENGTH
1	John	4
2	Jane	4
3	Alice	5

f. SUBSTR and INSTR

```
SELECT employee_id, SUBSTR(email, 1, INSTR(email, '@') - 1) AS username
FROM employees;
```

Output:

EMPLOYEE_ID	USERNAME
1	john.doe
2	jane.smith
3	alice.johnson

5. Queries Using Date Functions

a. SYSDATE

```
SELECT employee_id, first_name, SYSDATE AS current_date
FROM employees;
```

Output (example):

EMPLOYEE_ID	FIRST_NAME	CURRENT_DATE
1	John	10-OCT-2023
2	Jane	10-OCT-2023
3	Alice	10-OCT-2023

b. NEXT_DAY

```
SELECT employee_id, first_name, NEXT_DAY(hire_date, 'MONDAY') AS next_monday
FROM employees;
```

Output:

EMPLOYEE_ID	FIRST_NAME	NEXT_MONDAY
1	John	16-JAN-2023
2	Jane	21-NOV-2022
3	Alice	15-MAY-2023

c. ADD_MONTHS

```
SELECT employee_id, first_name, ADD_MONTHS(hire_date, 6) AS after_six_months
FROM employees;
```

Output:

EMPLOYEE_ID	FIRST_NAME	AFTER_SIX_MONTHS
1	John	15-JUL-2023
2	Jane	20-MAY-2023
3	Alice	10-NOV-2023

d. LAST_DAY

```
SELECT employee_id, first_name, LAST_DAY(hire_date) AS month_end
FROM employees;
```

Output:

EMPLOYEE_ID	FIRST_NAME	MONTH_END
1	John	31-JAN-2023
2	Jane	30-NOV-2022
3	Alice	31-MAY-2023

e. MONTHS_BETWEEN

```
SELECT employee_id, first_name, MONTHS_BETWEEN(SYSDATE, hire_date) AS
months_worked
FROM employees;
```

Output (example):

EMPLOYEE_ID	FIRST_NAME	MONTHS_WORKED
1	John	8.5
2	Jane	10.2
3	Alice	4.8

f. LEAST and GREATEST

```
SELECT employee_id, first_name, LEAST(hire_date, SYSDATE) AS earliest_date,  
GREATEST(hire_date, SYSDATE) AS latest_date  
FROM employees;
```

Output (example):

EMPLOYEE_ID	FIRST_NAME	EARLIEST_DATE	LATEST_DATE
1	John	15-JAN-2023	10-OCT-2023
2	Jane	20-NOV-2022	10-OCT-2023
3	Alice	10-MAY-2023	10-OCT-2023

g. TRUNC and ROUND

```
SELECT employee_id, first_name, TRUNC(hire_date, 'MONTH') AS truncated_date,  
ROUND(hire_date, 'MONTH') AS rounded_date  
FROM employees;
```

Output:

EMPLOYEE_ID	FIRST_NAME	TRUNCATED_DATE	ROUNDED_DATE
1	John	01-JAN-2023	01-JAN-2023
2	Jane	01-NOV-2022	01-DEC-2022
3	Alice	01-MAY-2023	01-MAY-2023

Program 5

- i. Create a simple PL/SQL program which includes declaration section, executable section and exception –Handling section (Ex. Student marks can be selected from the table and printed for those who secured first class and an exception can be raised if no records were found)

```
CREATE TABLE students
(
    id NUMBER PRIMARY KEY,
    name VARCHAR2(50),
    marks NUMBER
);
```

```
desc students;
```

Name	Null?	Type
-----	-----	-----
ID	NOT NULL	NUMBER
NAME		VARCHAR2(50)
MARKS		NUMBER

```
INSERT INTO students (id, name, marks) VALUES (1, 'John Doe', 75);
```

```
1 row created.
```

```
INSERT INTO students (id, name, marks) VALUES (2, 'Jane Doe', 58);
```

```
1 row created.
```

```
INSERT INTO students (id, name, marks) VALUES (3, 'Alex Smith', 62);
```

```
1 row created.
```

```
INSERT INTO students (id, name, marks) VALUES (4, 'Emily Johnson', 85);
```

```
1 row created.
```

```
Select * from students;
```

ID	NAME	MARKS
-----	-----	-----
1	John Doe	75
2	Jane Doe	58
3	Alex Smith	62
4	Emily Johnson	85

```
-- Declare section
```

```
DECLARE
```

```
    v_name VARCHAR2(20);
```

```
    v_marks NUMBER;
```

```
    no_records_found EXCEPTION;
```

```
-- Begin section
```

```

BEGIN
    -- Query to select students with first-class marks
    FOR rec IN (SELECT name, marks FROM students WHERE marks >=
60) LOOP
        v_name := rec.name;
        v_marks := rec.marks;
        DBMS_OUTPUT.PUT_LINE('Name: ' || v_name || ', Marks: ' ||
v_marks);
    END LOOP;

    -- Check if no records were found
    IF v_marks IS NULL THEN
        RAISE no_records_found;
    END IF;

    -- Exception section
EXCEPTION
    WHEN no_records_found THEN
        DBMS_OUTPUT.PUT_LINE('No records found for students with
first-class marks.');
```

```

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No data found in the table.');
```

```

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
```

```

END;
/
```

```

Name: John Doe, Marks: 75
Name: Alex Smith, Marks: 62
Name: Emily Johnson, Marks: 85
```

- ii. **Insert data into student table and use COMMIT, ROLLBACK and SAPOINT in PL/SQL block.**

```

BEGIN
    INSERT INTO students (id, name, marks) VALUES (1, 'John Doe', 75);
    INSERT INTO students (id, name, marks) VALUES (2, 'Jane Doe', 58);
    INSERT INTO students (id, name, marks) VALUES (3, 'Alex Smith',
62);
    INSERT INTO students (id, name, marks) VALUES (4, 'Emily Johnson',
85);
END;
/
Select * from students;
```

ID	NAME	MARKS
2	Jane Doe	58
1	John Doe	75
3	Alex Smith	62
4	Emily Johnson	85

```
-- Start a new transaction
BEGIN
  -- Insert a new record
  INSERT INTO students (id, name, marks) VALUES (5, 'Michael Brown',
90);

  -- Create a savepoint
  SAVEPOINT my_savepoint;

  -- Insert another record
  INSERT INTO students (id, name, marks) VALUES (6, 'Sarah Lee', 95);

  -- Rollback to the savepoint (undo the last insert)
  ROLLBACK TO my_savepoint;

  -- Commit the transaction (only the first insert will be committed)
  COMMIT;
END;
/
```

```
PL/SQL procedure successfully completed.
```

```
select * from students;
```

ID	NAME	MARKS
2	Jane Doe	58
1	John Doe	75
3	Alex Smith	62
4	Emily Johnson	85
5	Michael Brown	90

```
DECLARE
BEGIN
  -- Insert data into the student table
  INSERT INTO student (student_id, student_name, marks) VALUES (1, 'Alice', 85);
  INSERT INTO student (student_id, student_name, marks) VALUES (2, 'Bob', 55);
  INSERT INTO student (student_id, student_name, marks) VALUES (3, 'Charlie', 90);

  -- Savepoint after initial inserts
  SAVEPOINT after_inserts;
```

```

-- Insert more data
INSERT INTO student (student_id, student_name, marks) VALUES (4, 'David', 75);
INSERT INTO student (student_id, student_name, marks) VALUES (5, 'Eve', 45);

-- Rollback to the savepoint (undo the last two inserts)
ROLLBACK TO after_inserts;

-- Commit the changes (only the first three inserts will be saved)
COMMIT;

DBMS_OUTPUT.PUT_LINE('Data inserted and committed successfully.');
```

EXCEPTION

```

  WHEN OTHERS THEN
    -- Rollback in case of any error
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('An error occurred. Changes rolled back.');
```

END;

/

```
PL/SQL procedure successfully completed.
```

SQL> select * from student;

STUDENT_ID	STUDENT_NAME	MARKS
1	Alice	85
2	Bob	55
3	Charlie	90

Program 6

Develop a program that includes the features NESTED IF, CASE and CASE expression. The program can be extended using the NULLIF and COALESCE functions.

```
CREATE TABLE student
(
  student_id INT PRIMARY KEY,
  name VARCHAR(10),
  score INT,      -- Out of 100
  attendance INT  -- Days attended (out of 30)
);
```

```
desc student;
```

Name	Null?	Type
-----	-----	-----
STUDENT_ID	NOT NULL	NUMBER(38)
NAME		VARCHAR2(10)
SCORE		NUMBER(38)
ATTENDANCE		NUMBER(38)

```
BEGIN
INSERT INTO student VALUES(1, 'Alice', 92, 28);
INSERT INTO student VALUES (2, 'Bob', 75, 25);
INSERT INTO student VALUES (3, 'Charlie', 40, 0);
INSERT INTO student VALUES (4, 'Diana', NULL, 30);
INSERT INTO student VALUES (5, 'Evan', 82, NULL);
END;
/
```

```
PL/SQL procedure successfully completed.
```

```
select * from student;
```

STUDENT_ID	NAME	SCORE	ATTENDANCE
-----	-----	-----	-----
1	Alice	92	28
2	Bob	75	25
3	Charlie	40	0
4	Diana		30
5	Evan	82	

Query with Nested IF, CASE, NULLIF, and COALESCE

```
SELECT
  name,
  score,
  attendance,
```

```

-- Nested IF (simulated using CASE)
CASE
  WHEN score >= 90 THEN 'A'
  WHEN score >= 80 THEN 'B'
  WHEN score >= 70 THEN 'C'
  WHEN score >= 60 THEN 'D'
  ELSE 'F'
END AS grade,
-- CASE Expression (Searched CASE)
CASE
  WHEN COALESCE(score, 0) > 80 THEN 'High Performance'
  WHEN COALESCE(score, 0) BETWEEN 50 AND 80 THEN 'Medium Performance'
  ELSE 'Low Performance'
END AS performance_level,
-- NULLIF to avoid division by zero
(attendance / NULLIF(30, 0)) * 100 AS attendance_percentage,
-- COALESCE to handle NULL values
COALESCE(score, 0) AS adjusted_score
FROM student;

```

NAME	SCORE	ATTENDANCE	G	PERFORMANCE_LEVEL	ATTENDANCE_PERCENTAGE

ADJUSTED_SCORE					

Alice	92	28	A	High Performance	93.3333333
	92				
Bob	75	25	C	Medium Performance	83.3333333
	75				
Charlie	40	0	F	Low Performance	0
	40				

ADJUSTED_SCORE					

Diana	0	30	F	Low Performance	100
	0				
Evan	82		B	High Performance	
	82				

Program 7:

Write sql Program development using WHILE LOOPS, numeric FOR LOOPS, nested loops using ERROR Handling, BUILT –IN Exceptions, USE defined Exceptions, RAISE-APPLICATION ERROR.

```
Set serveroutput ON;
DECLARE
    -- Variables for loops
    v_num      NUMBER := 1;
    v_multiplier NUMBER := 1;

    -- User-defined exception
    e_number_too_high EXCEPTION;

BEGIN
    DBMS_OUTPUT.PUT_LINE('--- FOR LOOP: Printing numbers from 1 to 5 ---');

    -- Numeric FOR loop
    FOR i IN 1..5 LOOP
        DBMS_OUTPUT.PUT_LINE('Number: ' || i);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('--- WHILE LOOP: Multiplying numbers ---');

    -- WHILE loop
    WHILE v_num <= 5 LOOP
        DBMS_OUTPUT.PUT_LINE('Multiplication: ' || v_num || ' x ' || v_multiplier || ' = ' || (v_num * v_multiplier));
        v_num := v_num + 1;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('--- NESTED LOOP: Multiplication Table for 1 to 3 ---');

    -- Nested loops
    FOR i IN 1..3 LOOP
        FOR j IN 1..3 LOOP
            DBMS_OUTPUT.PUT_LINE(i || ' x ' || j || ' = ' || (i * j));
        END LOOP;
    END LOOP;

    -- Error Handling Demo
    BEGIN
        -- Intentional Division by zero
        DECLARE
            v_result NUMBER;
        BEGIN
```

```

        v_result := 100 / 0; -- This will cause ZERO_DIVIDE error
    EXCEPTION
        WHEN ZERO_DIVIDE THEN
            DBMS_OUTPUT.PUT_LINE('Error: Division by zero occurred!');
    END;

-- User-defined Exception Demo
DECLARE
    v_value NUMBER := 110;
BEGIN
    IF v_value > 100 THEN
        RAISE e_number_too_high;
    END IF;
EXCEPTION
    WHEN e_number_too_high THEN
        DBMS_OUTPUT.PUT_LINE('Error: Number exceeds allowed limit (User-
defined Exception)');
    END;

-- RAISE_APPLICATION_ERROR Demo
DECLARE
    v_age NUMBER := 15;
BEGIN
    IF v_age < 18 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Age should be 18 or above!');
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Custom Error: ' || SQLERRM);
    END;

END;

EXCEPTION
-- Built-in Exception Handler
WHEN VALUE_ERROR THEN
    DBMS_OUTPUT.PUT_LINE('Value error occurred!');
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Some unexpected error occurred: ' || SQLERRM);
END;
/

```

OUTPUT:

--- FOR LOOP: Printing numbers from 1 to 5 ---

Number: 1

Number: 2

Number: 3

Number: 4

Number: 5

--- WHILE LOOP: Multiplying numbers ---

Multiplication: $1 \times 1 = 1$

Multiplication: $2 \times 1 = 2$

Multiplication: $3 \times 1 = 3$

Multiplication: $4 \times 1 = 4$

Multiplication: $5 \times 1 = 5$

--- NESTED LOOP: Multiplication Table for 1 to 3 ---

$1 \times 1 = 1$

$1 \times 2 = 2$

$1 \times 3 = 3$

$2 \times 1 = 2$

$2 \times 2 = 4$

$2 \times 3 = 6$

$3 \times 1 = 3$

$3 \times 2 = 6$

$3 \times 3 = 9$

Error: Division by zero occurred!

Error: Number exceeds allowed limit (User-defined Exception)

Custom Error: ORA-20001: Age should be 18 or above!

Program 8:

write a SQL Programs development using creation of procedures, passing parameters IN and OUT of PROCEDURES.

```
CREATE TABLE Employees
```

```
(  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(15),  
    Department VARCHAR(10),  
    Salary DECIMAL(10,2)  
);
```

```
Desc Employees;
```

```
INSERT INTO Employees VALUES (101, 'Alice', 'HR', 50000);
```

```
INSERT INTO Employees VALUES (102, 'Bob', 'IT', 70000);
```

```
INSERT INTO Employees VALUES (103, 'Charlie', 'Finance', 60000);
```

```
Select * from Employees;
```

```
--Creating Procedure....
```

```
CREATE OR REPLACE PROCEDURE UpdateSalary (
```

```
    p_EmpID IN NUMBER,
```

```
    p_NewSalary IN NUMBER
```

```
)
```

```
AS
```

```
BEGIN
```

```
    UPDATE Employees
```

```
    SET Salary = p_NewSalary
```

```
    WHERE EmpID = p_EmpID;
```

```
    COMMIT;
```

```
END;
```

/

```
exec UpdateSalary(101,55000);
```

```
PL/SQL procedure successfully completed.
```

```
select * from employees;
```

EMPID	EMPNAME	DEPARTMENT	SALARY
101	Alice	HR	55000
102	Bob	IT	70000
103	Charlie	Finance	60000

Procedure with OUT Parameter:

```
CREATE OR REPLACE PROCEDURE GetSalary
```

```
(
```

```
    p_EmpID IN NUMBER,
```

```
    p_Salary OUT NUMBER
```

```
)
```

```
AS
```

```
BEGIN
```

```
    SELECT Salary INTO p_Salary
```

```
    FROM Employees
```

```
    WHERE EmpID = p_EmpID;
```

```
END;
```

```
/
```

```
Procedure created.
```

```
VARIABLE v_salary NUMBER;  
EXEC GetSalary(101, :v_salary);  
PRINT v_salary;
```

```
PL/SQL procedure successfully completed.  
  
SQL> PRINT v_salary;  
  
  V_SALARY  
-----  
      55000  
  
SQL> _
```

Program 9:

write a SQL plus oracle Program development using creation of stored functions, invoke functions in SQL Statements and write complex functions.

```
CREATE TABLE employees
(
    emp_id    NUMBER PRIMARY KEY,
    emp_name  VARCHAR2(10),
    salary    NUMBER(10, 2),
    dept_id   NUMBER
);
```

```
Table created.
```

```
INSERT INTO employees VALUES (1, 'John Doe', 50000, 10);
INSERT INTO employees VALUES (2, 'Jane Smith', 70000, 20);
INSERT INTO employees VALUES (3, 'Alan Brown', 45000, 10);
COMMIT;
```

```
Commit complete.
```

Simple Function to Calculate Annual Salary

```
CREATE OR REPLACE FUNCTION get_annual_salary(p_emp_id IN NUMBER)
RETURN NUMBER
IS
    v_annual_salary NUMBER;
BEGIN
    SELECT salary * 12
    INTO v_annual_salary
    FROM employees
    WHERE emp_id = p_emp_id;
    RETURN v_annual_salary;
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END;
/
```

```
Function created.
```

```
SELECT emp_name, get_annual_salary(emp_id) AS annual_salary
FROM employees;
```

EMP_NAME	ANNUAL_SALARY
John Doe	600000
Jane Smith	840000
Alan Brown	540000

Writing Complex Functions

```
CREATE OR REPLACE FUNCTION get_salary_grade(p_salary IN NUMBER)
RETURN VARCHAR2
IS
    v_grade VARCHAR2(10);
BEGIN
    IF p_salary >= 70000 THEN
        v_grade := 'A';
    ELSIF p_salary >= 50000 THEN
        v_grade := 'B';
    ELSE
        v_grade := 'C';
    END IF;

    RETURN v_grade;
END;
/
```

```
Function created.
```

```
SELECT emp_name, salary, get_salary_grade(salary) AS grade  
FROM employees;
```

EMP_NAME	SALARY
-----	-----
GRADE	
-----	-----
John Doe B	50000
Jane Smith A	70000
Alan Brown C	45000

Program 10:

Develop PL/SQL programs using features parameters in a CURSOR, FOR UPDATE CURSOR, WHERE CURRENT of clause and CURSOR variables.

```
CREATE TABLE employees (  
    emp_id    NUMBER PRIMARY KEY,  
    emp_name  VARCHAR2(15),  
    salary    NUMBER,  
    dept_id   NUMBER  
);
```

```
insert into employees values(1,'Alice',50000,101);  
insert into employees values(2,'Bob',60000,102);  
insert into employees values(3,'Charles',70000,103);  
insert into employees values(4,'David',80000,10);  
select * from employees;
```

EMP_ID	EMP_NAME	SALARY	DEPT_ID
1	Alice	50000	101
2	Bob	60000	102
3	Charles	70000	103
4	David	80000	10

--Parameterized Cursor

```
DECLARE  
    v_dept_id employees.dept_id%TYPE := 10;  
  
    CURSOR emp_cursor(p_dept_id NUMBER) IS  
        SELECT emp_id, emp_name, salary  
        FROM employees  
        WHERE dept_id = p_dept_id;  
  
    v_emp_id   employees.emp_id%TYPE;  
    v_emp_name employees.emp_name%TYPE;  
    v_salary   employees.salary%TYPE;  
BEGIN  
    OPEN emp_cursor(v_dept_id);  
    LOOP  
        FETCH emp_cursor INTO v_emp_id, v_emp_name, v_salary;  
        EXIT WHEN emp_cursor%NOTFOUND;
```

```

        DBMS_OUTPUT.PUT_LINE('ID: ' || v_emp_id || ', Name: ' || v_emp_name || ',
Salary: ' || v_salary);
    END LOOP;
    CLOSE emp_cursor;
END;
/
ID: 4, Name: David, Salary: 80000
PL/SQL procedure successfully completed.

```

-- 2. FOR UPDATE Cursor

```

DECLARE
    CURSOR emp_cursor IS
        SELECT emp_id, salary
        FROM employees
        WHERE dept_id = 10
        FOR UPDATE;

    v_emp_id employees.emp_id%TYPE;
    v_salary employees.salary%TYPE;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO v_emp_id, v_salary;
        EXIT WHEN emp_cursor%NOTFOUND;

        -- Increase salary by 10%
        UPDATE employees
        SET salary = v_salary * 1.1
        WHERE emp_id = v_emp_id;
    END LOOP;
    CLOSE emp_cursor;
END;
/
PL/SQL procedure successfully completed.

```

```
select * from employees;
```

EMP_ID	EMP_NAME	SALARY	DEPT_ID
1	Alice	50000	101
2	Bob	60000	102
3	Charles	70000	103
4	David	88000	10

11. Develop Programs using BEFORE and AFTER Triggers, Row and Statement Triggers and INSTEAD OF Triggers

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary DECIMAL(10, 2),  
    department_id INT  
);
```

```
CREATE TABLE departments (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(50)  
);
```

```
CREATE TABLE employee_audit (  
    audit_id SERIAL PRIMARY KEY,  
    emp_id INT,  
    action_type VARCHAR(10),  
    changed_by VARCHAR(50),  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

BEFORE and AFTER Triggers

BEFORE INSERT Trigger: Validate salary before insert

```
CREATE OR REPLACE TRIGGER before_emp_insert  
BEFORE INSERT ON employees  
FOR EACH ROW  
BEGIN  
    IF :NEW.salary < 10000 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Salary must be at least 10000');  
    END IF;  
END;
```

AFTER INSERT Trigger: Log insert in audit table

```
CREATE OR REPLACE TRIGGER after_emp_insert  
AFTER INSERT ON employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO employee_audit (emp_id, action_type, changed_by)  
    VALUES (:NEW.emp_id, 'INSERT', USER);  
END;
```

2. ROW and STATEMENT Level Triggers

ROW Level Trigger: AFTER UPDATE on employees

```
CREATE OR REPLACE TRIGGER after_emp_update_row
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit (emp_id, action_type, changed_by)
    VALUES (:NEW.emp_id, 'UPDATE', USER);
END;
```

STATEMENT Level Trigger: AFTER UPDATE

```
CREATE OR REPLACE TRIGGER after_emp_update_stmt
AFTER UPDATE ON employees
BEGIN
    DBMS_OUTPUT.PUT_LINE('Employees table updated');
END;
```

2. **INSTEAD OF Trigger** (Used with Views)

```
CREATE OR REPLACE VIEW emp_dept_view AS
SELECT e.emp_id, e.emp_name, e.salary, d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id;
```

12. write a sql program to Create a table and perform the search operation on table using indexing and non- indexing techniques.

```
CREATE TABLE customers
(
  customer_id NUMBER PRIMARY KEY,
  customer_name VARCHAR2(100),
  email VARCHAR2(100),
  city VARCHAR2(50)
);
```

Insert Sample Data:

```
BEGIN
  FOR i IN 1..10000 LOOP
    INSERT INTO customers (customer_id, customer_name, email, city)
    VALUES (
      i,
      'Customer_' || i,
      'customer_' || i || '@example.com',
      CASE
        WHEN MOD(i, 3) = 0 THEN 'New York'
        WHEN MOD(i, 3) = 1 THEN 'Los Angeles'
        ELSE 'Chicago'
      END
    );
  END LOOP;
  COMMIT;
END;
```

Search (Without Index)

```
-- This query does not use any index initially (full table scan)
SELECT * FROM customers
WHERE city = 'Chicago';
```

Analyze Execution Plan (Before Index)

```
EXPLAIN PLAN FOR
SELECT * FROM customers
WHERE city = 'Chicago';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Create Index

```
CREATE INDEX idx_city ON customers(city);
```

Search (With Index)

```
-- Now the same query may use the index  
SELECT * FROM customers  
WHERE city = 'Chicago';
```

Analyze Execution Plan (After Index):

```
EXPLAIN PLAN FOR  
SELECT * FROM customers  
WHERE city = 'Chicago';  
  
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

13. Write a Java program that connects to a database using JDBC

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class OracleJDBCConnection
{

    public static void main(String[] args)
    {

        // JDBC connection parameters for Oracle
        String url = "jdbc:oracle:thin:@localhost:1521:XE"; // Host:Port:SID
        String user = "system"; // replace with your Oracle username
        String password = "your_password"; // replace with your Oracle password

        Connection connection = null;

        try {
            // Load Oracle JDBC Driver
            Class.forName("oracle.jdbc.driver.OracleDriver");

            // Establish connection
            connection = DriverManager.getConnection(url, user, password);
            System.out.println("Connected to Oracle Database!");

        } catch (ClassNotFoundException e) {
            System.out.println(" Oracle JDBC Driver not found.");
            e.printStackTrace();

        } catch (SQLException e) {
            System.out.println(" Failed to connect to Oracle DB.");
            e.printStackTrace();

        } finally {
            // Close the connection
            try {
                if (connection != null && !connection.isClosed()) {
                    connection.close();
                    System.out.println("🔒 Connection closed.");
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

14. Write a Java program to connect to a database using JDBC and insert values into it.

```
CREATE TABLE students (  
    student_id NUMBER PRIMARY KEY,  
    student_name VARCHAR2(100),  
    department VARCHAR2(50)  
);
```

Java Code to Connect and Insert Data

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.PreparedStatement;  
import java.sql.SQLException;  
  
public class OracleJDBCInsertExample  
{  
  
    public static void main(String[] args)  
    {  
        String url = "jdbc:oracle:thin:@localhost:1521:XE"; // Replace XE with your  
SID  
        String user = "system"; // Replace with your username  
        String password = "your_password"; // Replace with your password  
  
        Connection conn = null;  
        PreparedStatement pstmt = null;  
  
        try {  
            // Load Oracle JDBC Driver  
            Class.forName("oracle.jdbc.driver.OracleDriver");  
  
            // Establish connection  
            conn = DriverManager.getConnection(url, user, password);  
            System.out.println(" Connected to Oracle DB!");  
  
            // Insert query using PreparedStatement  
            String insertSQL = "INSERT INTO students (student_id, student_name,  
department) VALUES (?, ?, ?)";  
            pstmt = conn.prepareStatement(insertSQL);  
  
            // Set values  
            pstmt.setInt(1, 1);  
            pstmt.setString(2, "Alice");  
            pstmt.setString(3, "Computer Science");  
        }  
    }  
}
```

```

        // Execute update
        int rowsInserted = pstmt.executeUpdate();
        System.out.println(" " + rowsInserted + " row(s) inserted.");

    } catch (ClassNotFoundException e) {
        System.out.println(" Oracle JDBC Driver not found.");
        e.printStackTrace();

    } catch (SQLException e) {
        System.out.println(" SQL error occurred.");
        e.printStackTrace();

    } finally {
        try {
            if (pstmt != null) pstmt.close();
            if (conn != null) conn.close();
            System.out.println("🔒 Connection closed.");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
}

```

15. Write a Java program to connect to a database using JDBC and delete values from it.

```
CREATE TABLE students (  
    student_id NUMBER PRIMARY KEY,  
    student_name VARCHAR2(100),  
    department VARCHAR2(50)  
);
```

Java Program: Delete by student_id

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.PreparedStatement;  
import java.sql.SQLException;  
  
public class OracleJDBCDeleteExample {  
  
    public static void main(String[] args) {  
        // Oracle DB connection settings  
        String url = "jdbc:oracle:thin:@localhost:1521:XE"; // Replace XE with your SID  
        if different  
        String user = "system"; // Replace with your Oracle username  
        String password = "your_password"; // Replace with your password  
  
        Connection conn = null;  
        PreparedStatement pstmt = null;  
  
        try {  
            // Load Oracle JDBC Driver  
            Class.forName("oracle.jdbc.driver.OracleDriver");  
  
            // Connect to Oracle  
            conn = DriverManager.getConnection(url, user, password);  
            System.out.println("Connected to Oracle DB!");  
  
            // SQL DELETE statement  
            String deleteSQL = "DELETE FROM students WHERE student_id = ?";  
  
            // Create PreparedStatement  
            pstmt = conn.prepareStatement(deleteSQL);  
  
            // Set value (e.g., delete student with ID 2)  
            pstmt.setInt(1, 2);  
  
            // Execute delete  
            int rowsDeleted = pstmt.executeUpdate();
```

```

        if (rowsDeleted > 0) {
            System.out.println(" Successfully deleted " + rowsDeleted + " record(s).");
        } else {
            System.out.println(" No matching record found to delete.");
        }

    } catch (ClassNotFoundException e) {
        System.out.println(" JDBC Driver not found.");
        e.printStackTrace();
    } catch (SQLException e) {
        System.out.println(" SQL Error occurred.");
        e.printStackTrace();
    } finally {
        try {
            if (pstmt != null) pstmt.close();
            if (conn != null) conn.close();
            System.out.println(" Connection closed.");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
}
}

```